

ModelForm validation

models.py:--

```
from django.db import models
import re
from django.core.exceptions import ValidationError

## ----- Inline built-in validators-----
# from django.core.validators import
MinLengthValidator,MaxValueValidator,FileExtensionValidator,MaxLengthValidator
# from django.core.validators import MinValueValidator, RegexValidator,
EmailValidator,MaxValueValidator

# # Create your models here.

# class Student(models.Model):
#     name = models.CharField(max_length=50,validators=[MinLengthValidator(3),
#                                                         MaxLengthValidator(10),
#                                                         RegexValidator(regex=r'^[A-Za-z]+$',
#                                                         message='Only alphabets
are allowed')])
#     age = models.IntegerField(validators=[MinValueValidator(18),
MaxValueValidator(60)])
#     email = models.EmailField(validators=[EmailValidator()])
#     phone = models.CharField(max_length=10,validators=[
RegexValidator(regex=r'^[6-9]\d{9}$',
                                                         message='Enter a valid
Indian mobile number')])
#     resume = models.FileField(upload_to='resumes/',validators=[
FileExtensionValidator(allowed_extensions=['pdf', 'docx'])])

#     def save(self, *args, **kwargs): # save required, when we called
Model.objects.create()
#         self.full_clean()
#         super().save(*args, **kwargs)

# # ----- Inline custom-validation -----
# from django.db import models

# # Create your models here.
```

```

# def validate_name(value):
#     if len(value) < 3 or len(value) > 10:
#         raise ValidationError("Name must be between 3 and 10 characters")
#     if not value.isalpha():
#         raise ValidationError("Name must contain only alphabets")

# def validate_age(value):
#     if value < 18 or value > 60:
#         raise ValidationError("Age must be between 18 and 60")

# def validate_email(value):
#     if not value.endswith("@gmail.com"):
#         raise ValidationError("Only Gmail addresses are allowed")

# def validate_phone(value):
#     if not re.match(r'^[6-9]\d{9}$', value):
#         raise ValidationError("Enter a valid Indian mobile number")

# def validate_resume(file):
#     if not file.name.endswith(('.pdf', '.docx')):
#         raise ValidationError("Only PDF or DOCX files are allowed")
#     if file.size > 2 * 1024 * 1024:
#         raise ValidationError("Resume size must be under 2 MB")

# class Student(models.Model):
#     name = models.CharField(max_length=50, validators=[validate_name])
#     age = models.IntegerField(validators=[validate_age])
#     email = models.EmailField(validators=[validate_email])
#     phone = models.CharField(max_length=10, validators=[validate_phone])
#     resume = models.FileField(upload_to='resumes/', validators=[validate_resume])

#     def save(self, *args, **kwargs): # save required, when we called
#         Model.objects.create()
#         self.full_clean()
#         super().save(*args, **kwargs)

## -----Model-Level custom-validation-----
class Student(models.Model):
    name = models.CharField(max_length=50)
    age = models.IntegerField()
    email = models.EmailField()
    phone = models.CharField(max_length=10)
    resume = models.FileField(upload_to='resumes/')

```

```

def clean(self):
    errors = {}
    # Name validation
    if not self.name.isalpha():
        errors['name'] = "Name must contain only alphabets"
    elif len(self.name) < 3 or len(self.name) > 10:
        errors['name'] = "Name must be between 3 and 10 characters"

    # Age validation
    if self.age < 18 or self.age > 60:
        errors['age'] = "Age must be between 18 and 60"

    # Email validation
    if not self.email.endswith("@gmail.com"):
        errors['email'] = "Only Gmail email allowed"

    # Phone validation
    if not re.match(r'^[6-9]\d{9}$', self.phone):
        errors['phone'] = "Enter a valid Indian mobile number"

    # Resume validation
    if self.resume:
        if not self.resume.name.endswith(('.pdf', '.docx')):
            errors['resume'] = "Only PDF or DOCX files allowed"
        elif self.resume.size > 2 * 1024 * 1024:
            errors['resume'] = "Resume size must be under 2 MB"

    # Raise all errors together
    if errors:
        raise ValidationError(errors)

def save(self, *args, **kwargs):
    self.full_clean() # agar aap clean() ke baad save() override karke usme
    # full_clean() call karte ho, to Model.objects.create() par validation hoga
    super().save(*args, **kwargs)

```

forms.py:-

```

from django import forms
from .models import Student

class StudentForm(forms.ModelForm):
    class Meta:

```

```
model = Student
fields = ['name', 'age', 'email', 'phone', 'resume']
# fields = '__all__'
```

urls.py:-

```
"""
```

URL configuration for project project.

The `urlpatterns` list routes URLs to views. For more information please see:
<https://docs.djangoproject.com/en/6.0/topics/http/urls/>

Examples:

Function views

1. Add an import: `from my_app import views`
2. Add a URL to `urlpatterns`: `path('', views.home, name='home')`

Class-based views

1. Add an import: `from other_app.views import Home`
2. Add a URL to `urlpatterns`: `path('', Home.as_view(), name='home')`

Including another URLconf

1. Import the `include()` function: `from django.urls import include, path`
2. Add a URL to `urlpatterns`: `path('blog/', include('blog.urls'))`

```
"""
```

```
from django.contrib import admin
from django.urls import path
from django.conf import settings
from django.conf.urls.static import static
from app import views
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', views.student_create, name='student_create'),
    path('success/', views.success, name='success'),
] + static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

views.py:-

```
from django.shortcuts import render, redirect
from .forms import StudentForm
```

Create your views here.

```
def student_create(request):
    if request.method == "POST":
        form = StudentForm(request.POST, request.FILES)
        if form.is_valid():
            form.save()
```

```

        return redirect("success")
    else:
        form = StudentForm()

    return render(request, "student_form.html", {"form": form})

def success(request):
    return render(request, "success.html")

```

app/templates/student_form.html:-

```

<!DOCTYPE html>
<html lang="en">
<head>
    <title>Student Form</title>

    <!-- Bootstrap 5 CSS -->
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>

<body class="bg-light">

<div class="container d-flex justify-content-center align-items-center vh-100">
    <div class="card shadow p-4 w-50">

        <h2 class="text-center mb-4">Student Form</h2>

        <form method="post" enctype="multipart/form-data">
            {% csrf_token %}

            <!-- Name -->
            <label>Name:</label>
            {{ form.name }}
            <div style="color:red;"> {{ form.name.errors }} </div>

            <!-- Age -->
            <label>Age:</label>
            {{ form.age }}
            <div style="color:red;"> {{ form.age.errors }} </div>

            <!-- Email -->

```

```
<label>Email:</label>
{{ form.email }}
<div style="color:red;"> {{ form.email.errors }} </div>

<!-- Phone -->
<label>Phone:</label>
{{ form.phone }}
<div style="color:red;"> {{ form.phone.errors }} </div>

<!-- Resume -->
<label>Resume:</label>
{{ form.resume }}
<div style="color:red;"> {{ form.resume.errors }} </div>
<br>
<button type="submit">Submit</button>
</form>
</div>
</div>
</body>
</html>
```

app/templates/success.html:

Data save Successfully!